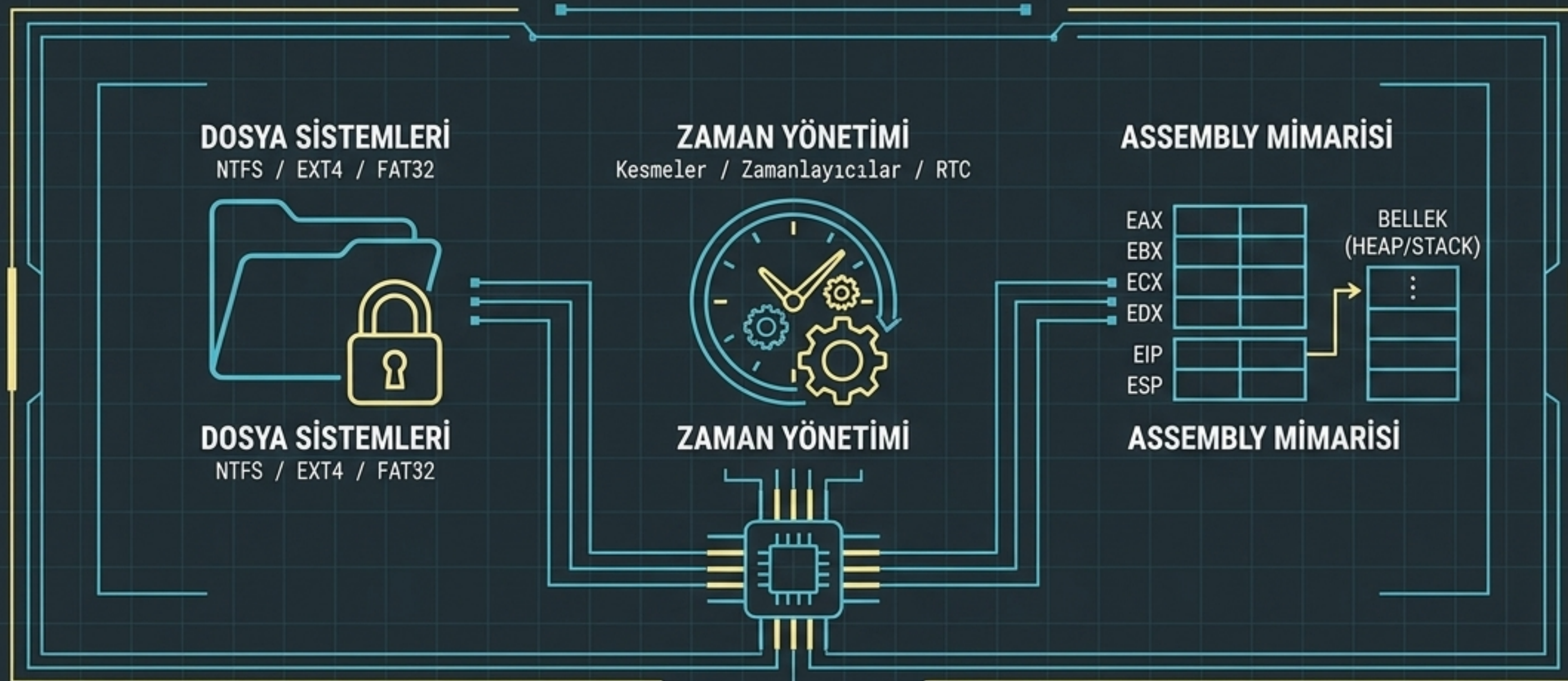


Sistem Programlama: Nihai Sınav Kılavuzu

Dosya Sistemleri, Zaman Yönetimi ve Assembly Mimarisi (Blueprint Sentezi)



Umask Mantığı ve İzin Hesaplama

Umask, dosya oluşturulurken 'fazla yetkileri' tıraşlayan bir filtredir.

$$\text{gerçek_mode} = \text{mode} \& \sim \text{umask}$$

$\& \sim$
(Bitsel VE DEĞİL)

Mode:	0666	(Varsayılan Dosya İzni)
Umask:	0022	(Kapatılacak İzinler)
Sonuç:	0644	(Gerçek İzin: rw- r-- r--)

[Sınav İpucu]: Sistem çağrılarında (C kodu) umask oktal (sekizlik) belirtilmelidir (ör. 022). Terminalde ilk sıfıra gerek yoktur.

Umask Sık Karşılaşılan Senaryolar Matrisi

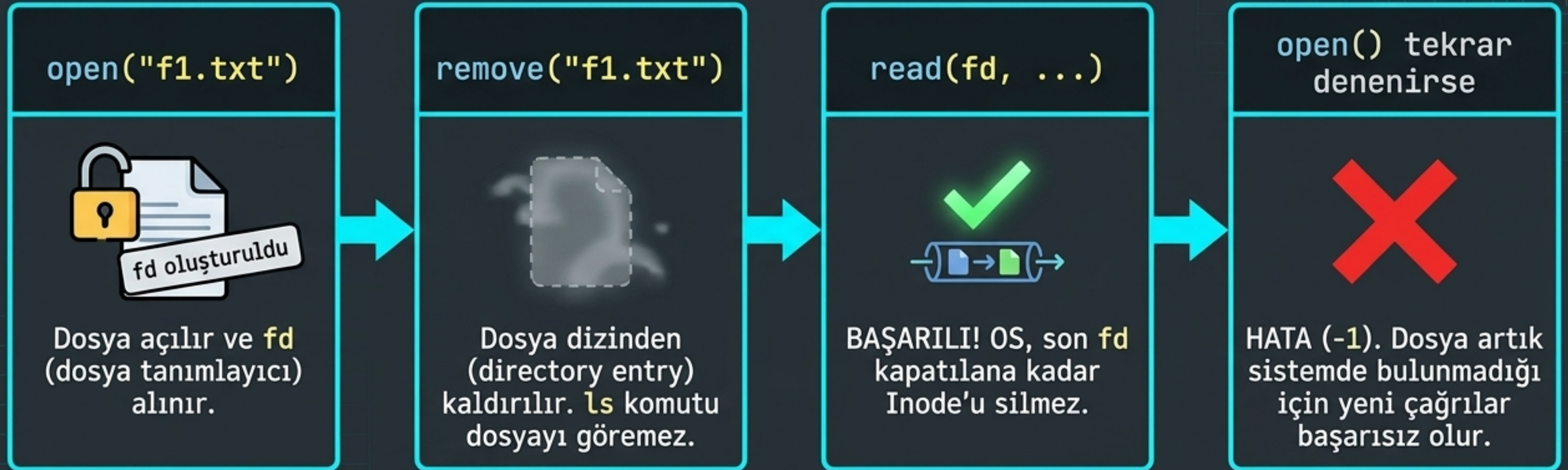
Umask Değeri	Kapatılan Bitler	Dosya Sonuç (Base: 0666)	Dizin Sonuç (Base: 0777)
000	Hiçbiri	0666 (rw-rw-rw-)	0777 (rwxrwxrwx)
022	Group & World (Write)	0644 (rw-r--r--)	0755 (rwxr-xr-x)
077	Tüm Group & World	0600 (rw-----)	0700 (rwx-----)
0777	Dokuz bitin tümü	0000 (-----)	0000 (d-----)



Dikkat: `open()` çağrısı geçerli bir 'file descriptor' (fd) döndürdüyse, dosya izni daha sonra 0000 yapılsa bile DOSYA AÇIK OLDUĞU SÜRECE mevcut proses yazmaya devam edebilir!

Hileli Sınav Senaryosu: Açık Dosyanın Silinmesi

İşletim sistemi dosyaları ne zaman GERÇEKTEN siler?



Dikkat: Dosya açık kaldığı sürece disk alanı serbest bırakılmaz!

OS, dosya inode'unu ve veri bloklarını sadece tüm dosya tanımlayıcıları kapatıldığında gerçekten siler.

Dosya İşlemleri Çeviri Tablosu

C Sistem Çağrıları ve UNIX Shell Karşılıkları

C Sistem Çağrısı	UNIX Komutu
link(f1, f2) <=====>	ln f1 f2 (Sadece dosyalar, dizin olamaz)
unlink(f1) <=====>	rm f1 (Dizinlerde başarısız olur)
remove(f1) <=====>	rm f1 (Boş dizinleri de silebilir)
rename(f1, f2) <=====>	mv f1 f2

Soru 1 Çözüm Stratejisi Paneli

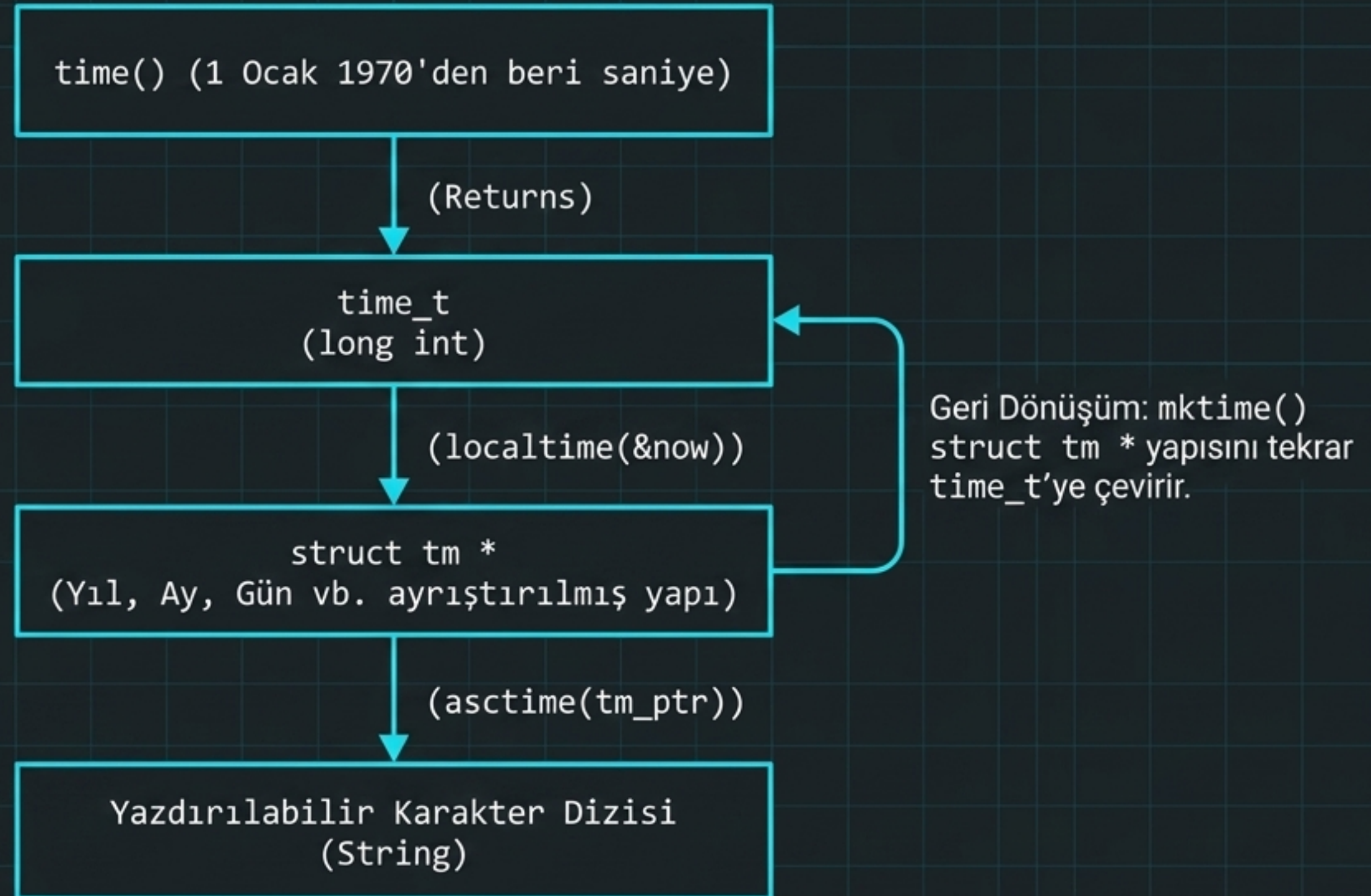
Soru: Dizindeki dosyaları/alt dizinleri aç ve readonly yap?

Algoritma Adımları:

1. **opendir()** / **readdir()** ile dizini tara.
2. **stat()** ile dosya/dizin tipini kontrol et.
3. **chmod(path, 0444)** (Dosyalar için salt okunur).
4. **chmod(path, 0555)** (Dizinler için: okuma + içine girme yetkisi [execute]).

Zaman Yönetimi Veri Boru Hattı (Pipeline)

Zaman veri tipleri arasındaki dönüşüm şeması.



Kritik Hata Noktası: localtime() her zaman AYNI adresi (pointer) döndürür (malloc kullanılmaz). Ardışık çağrılarda önceki değer üzerine yazar!

CPU Kayıtçıları (Registers) Mimarisi

Jassem Sanal Makinesi 32-bit Kayıtçı Sınıflandırması

Genel Amaçlı (Kullanıcı R/W)

`r0, r1, r2, r3, r4`
(Veri ve matematiksel işlemler için)

İşaretçiler (Pointers)

`sp` (Stack Pointer / Yığın İşaretçisi)
`fp` (Frame Pointer / Çerçeve İşaretçisi)
`pc` (Program Counter / Sıradaki Komut Adresi)

Salt Okunur (Read-Only)

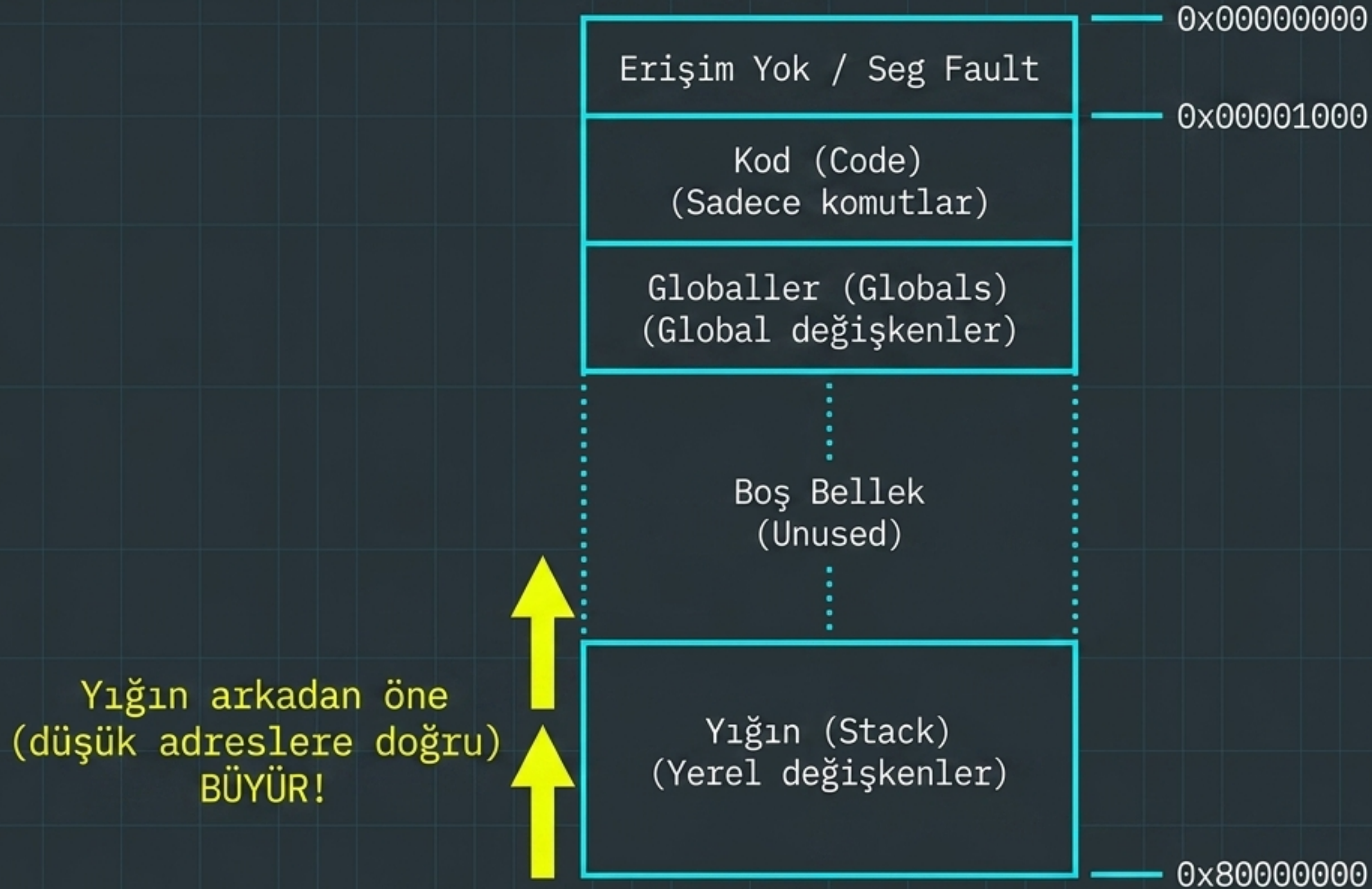
`g0` (Her zaman 0) | `g1` (Her zaman 1) |
`gm1` (Her zaman -1)

Gizli (Kullanıcı Erişemez)

`IR` (Komut Kayıtçısı - Çalışan komut)
`CSR` (Durum Kayıtçısı)

Bellek Adres Alanı (Address Space)

Bir programın belleğe mantıksal bakışı (0x80000000 boyutu).



Assembly Bellek ve Kontrol Komutları Kılavuzu

Panel 1: Bellek İşlemleri

ld mem -> %reg : Yükle (Load)
st %reg -> mem : Kaydet (Store)

Adresleme Tipleri:

st %r0 -> i (Global değişkene kaydet)
st %r0 -> [r1] (r1'deki değeri
adres/pointer kabul et)
st %r0 -> [fp+4] (Çerçeve işaretçisi
ofseti ile erişim)
st %r0 -> [sp]-- (Değeri kaydet ve
sp'yi azalt)

Panel 2: Matematik & Kontrol

mov #val -> %reg : Sabit değer ata

add / sub / mul / idiv / imod
%reg1, %reg2 -> %reg3 (Matematik)

push #val / pop #val : sp'den çıkar /
sp'ye ekle

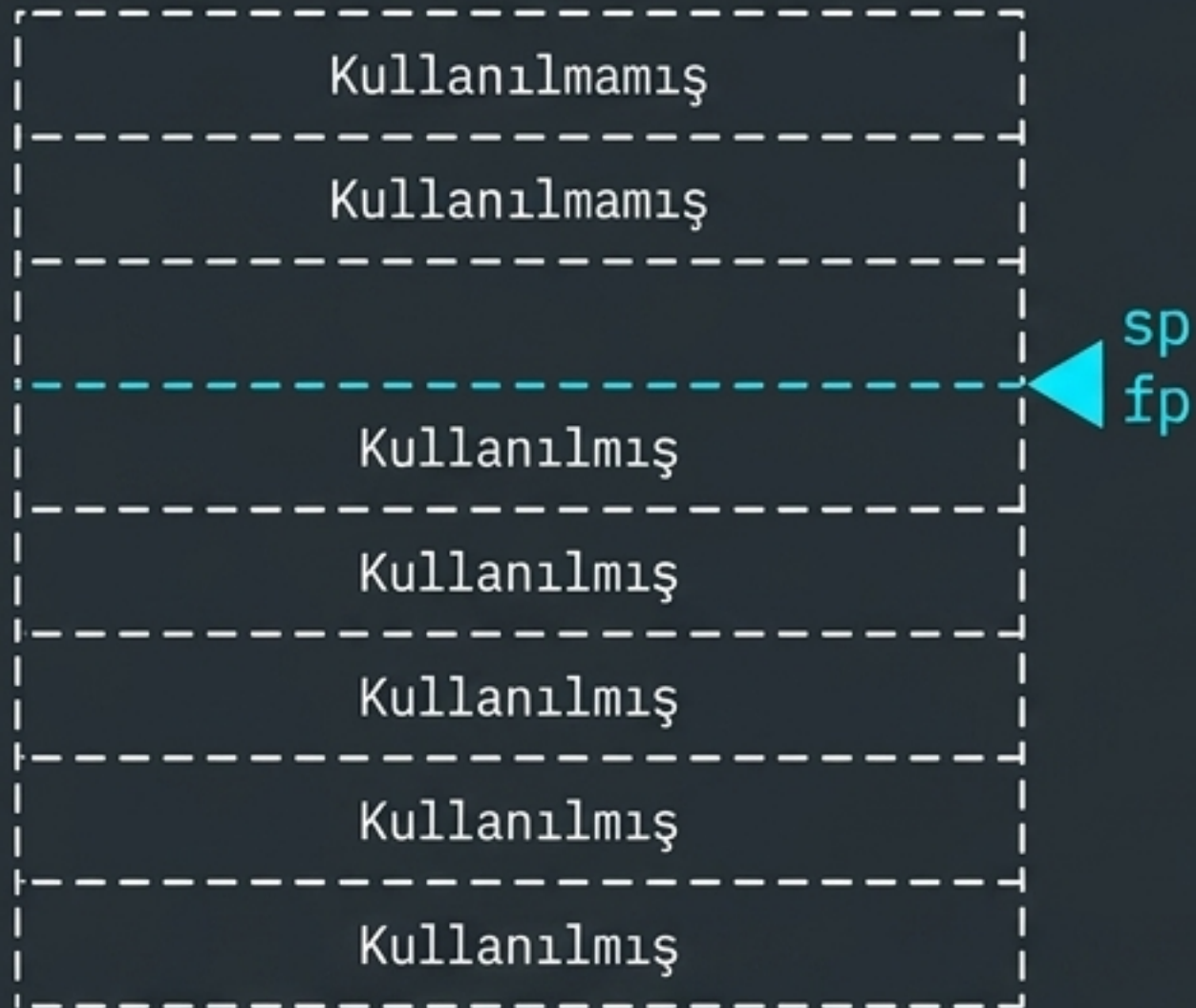
jsr a : Alt program çağır (Jump to
Subroutine)

ret : Geri dön (Return)

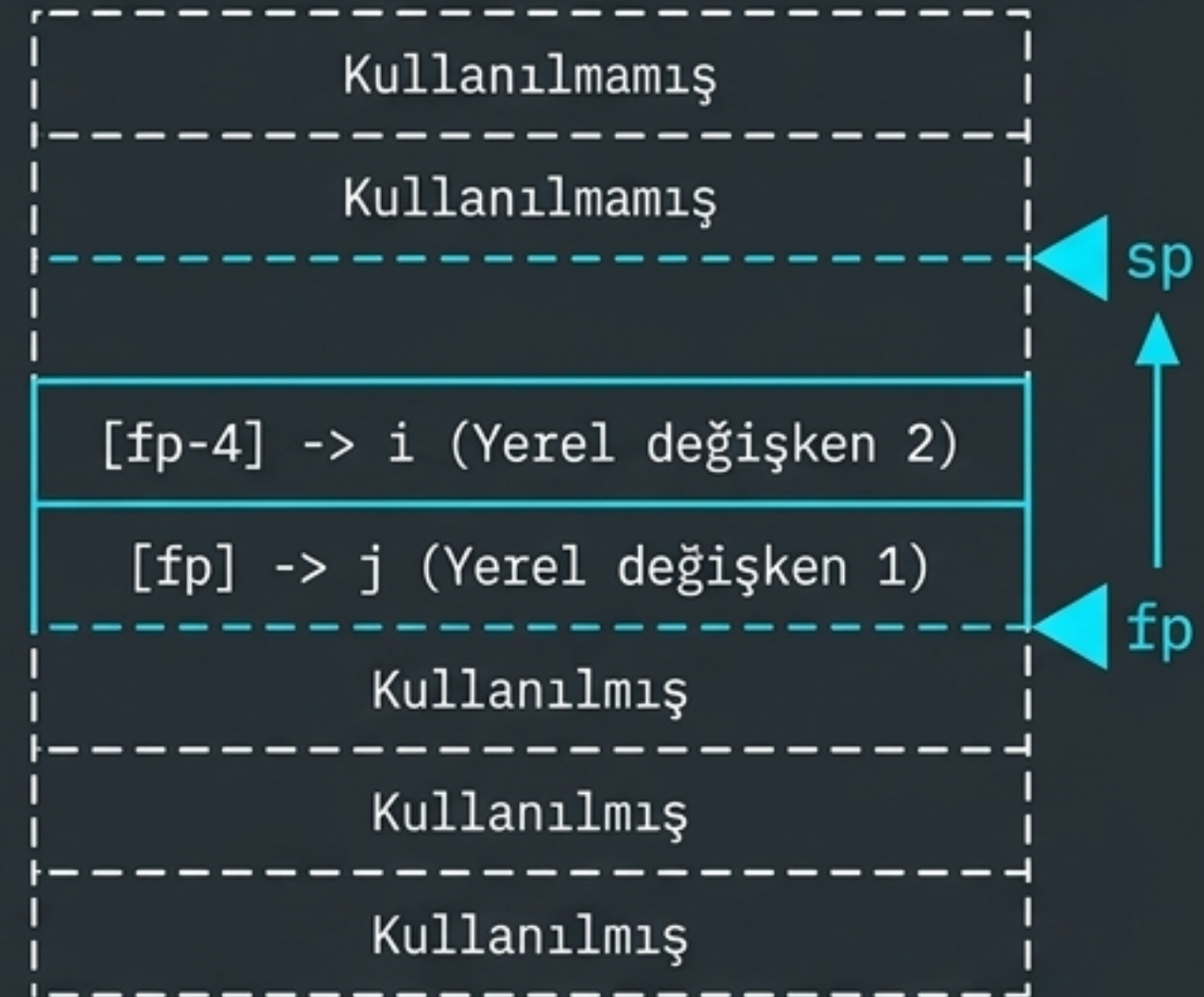
Yığın Çerçeveleri (Stack Frames) ve Tahsis

`sp` (Üst Taraf) ve `fp` (Alt Taraf) bellek sınırlarını çizer.

Adım 1: Boş Çerçeve



Adım 2: push #8 Sonrası



Yığın işaretçisini (`sp`) 8 azaltmak (push #8),
iki adet 4 baytlık tamsayı (int) tahsis eder.

Kod Sentezi: C'den Assembly'e Çeviri

Global değişken kullanımı (Derleyici optimizasyonu kapalı).

Sol Panel: C Kodu

```
int i;  
int j;  
int main() {  
    i = 3;  
    j = 2;  
    j = i + j;  
}
```

Sağ Panel: Jassem Assembly

```
.globl i  
.globl j  
mov #3 -> %r0  
st %r0 -> i  
mov #2 -> %r0  
st %r0 -> j  
{  
    ld i -> %r0  
    ld j -> %r1  
    add %r0,%r1 -> %r1  
    st %r1 -> j  
}
```


Final Sınavı Soru Çözümü

Karmaşık Tahsis (Global + Yerel Değişkenler Bir Arada)

Soru C Kodu

```
int i;

main(){
    int j, k;
    i=3;
    j=5;
    k=i+j;
    i++;
}
```

Bellek Stratejisi

1. i global -> **.globl i**
2. j ve k yerel -> **push #8**
(sp ve fp ile erişim)
3. j -> **[fp]**, k -> **[fp-4]**

Assembly Karşılığı

```
.globl i

main:
    push #8                / j ve k için yer ayır
    mov #3 -> %r0
    st %r0 -> i             / i = 3
    mov #5 -> %r0
    st %r0 -> [fp]          / j = 5

    ld i -> %r0
    ld [fp] -> %r1
    add %r0,%r1 -> %r1
    st %r1 -> [fp-4]        / k = i + j

    ld i -> %r0
    add %r0, g1 -> %r0      / i++ (g1 = 1 sabitini kullan)
    st %r0 -> i
    ret
```